

CSCE 628 COMPUTATIONAL BIOLOGY · MODULE 07 · LECTURE NOTES

Hidden Markov Models

Alignment's recurrence, with probabilities and hidden state.

MODULE OUTCOMES

- Define an HMM and its three standard problems.
- Derive Viterbi and the forward algorithm.
- Explain profile HMMs and remote homology detection.
- Explain gene finding and its limits.
- Implement and apply an HMM.

1 The model

states S , transitions $a(i,j)$, emissions $e_i(x)$, initial distribution π

- A Markov chain over hidden states, each of which emits an observable symbol — you see the symbols and the states are what you want, which is the whole setup.
- **Decoding:** find the single most likely state path — **argmax over paths of $P(\text{path} \mid \text{sequence})$** — which is the **Viterbi algorithm**, and is dynamic programming with exactly the max-over-predecessors structure of Module 02.
- **Evaluation:** compute $P(\text{sequence})$, summed over all paths — the forward algorithm, which is the same recurrence with a sum in place of the max — and gives the likelihood, which is what scoring and model comparison need (a single best path is not a likelihood).
- **Viterbi is max and forward is sum, over the same recurrence** — which is the entire relationship between the two, and once seen it cannot be unseen. **If you can write Needleman-Wunsch you can write Viterbi**, which is why this module is shorter than its importance suggests.

AND THE THIRD PROBLEM IS LEARNING THE PARAMETERS, WHICH IS WHERE IT GETS DELICATE

Given sequences and no state labels, Baum-Welch (an instance of expectation-maximisation) estimates the transition and emission parameters — and converges to a local optimum (CSCE 633 Module 07's EM material, with the same caveat).

So initialisation matters, and different starting points give genuinely different models — which should be reported and usually is not, exactly as with any non-convex fit (CSCE 669 Module 04).

And with labelled data the estimation is just counting, with pseudocounts added to avoid zero probabilities that would make unseen events impossible rather than rare — which is far more reliable, and is what profile HMMs do (§3), since a curated family alignment supplies the labels.

Which is the general preference this program keeps arriving at: estimate from labelled examples wherever you have them, and treat unsupervised structure discovery as the weaker claim (CSCE 676 Module 05's clustering caution).

2 Viterbi

```
v_k(i) = e_k(x_i) * max over j of [ v_j(i-1) * a(j,k) ]
```

`v_k(i)` the best probability of any path ending
in state `k` having emitted the first `i`
symbols
traceback pointer: which `j` achieved the max

COMPARE Module 02:

max over three predecessors	-> max over states
substitution score	-> emission prob
gap penalty	-> transition prob
SAME SHAPE.	

TWO RULES FOR IMPLEMENTING IT

- 1 work in log space. Products of thousands of probabilities underflow to zero in double precision, silently and without warning.
- 2 for the forward algorithm, summing in log space needs a log-sum-exp with the maximum factored out, or it overflows instead.

THESE TWO BUGS ACCOUNT FOR MOST BROKEN HMM CODE.

Work in log space, and use log-sum-exp for the forward algorithm — these two bugs account for most broken HMM implementations, and both fail silently: the underflow produces zeros that propagate, and the naive log-space sum produces infinities. Both are caught immediately by testing on a sequence long enough to underflow, which a toy example will not be.

3 Profile HMMs

A PROFILE HMM ENCODES A FAMILY'S POSITION-SPECIFIC CONSERVATION, WHICH IS STRICTLY MORE INFORMATION THAN ANY ONE SEQUENCE

Match, insert, and delete states for each column of a family alignment — with position-specific emission probabilities and position-specific gap probabilities, all estimated by counting from the alignment with pseudocounts.

So a position that is absolutely conserved across the family penalises a mismatch heavily, while a variable position penalises it barely at all — which a single global substitution matrix simply cannot express, since it has one score per residue pair regardless of position (Module 03 §1).

Which is why profile methods find remote homologues that pairwise BLAST misses (Module 04 §1's concentrated misses) — **the information is in the family, not in any individual member**, and no pairwise comparison can recover it.

And iterated search builds the profile as it goes, adding each round's hits and re-searching — **which is powerful, and which can drift: a false hit incorporated into the profile recruits more sequences like itself**, and the profile walks away from the original family. **This is a specific, known, and still common failure**, and is why inclusion thresholds matter.

4 Gene finding

- **States for coding regions, introns, intergenic sequence, and the signal sites (start, stop, splice donors and acceptors) — and Viterbi labels every base of the input**, producing a complete and internally plausible annotation of the whole sequence.
- **It is very good at this, and it is how most genome annotation starts — which is exactly why the caveats matter**: a method this useful gets its output treated as data.
- **But Viterbi always returns a path — the most likely one, however unlikely in absolute terms — so it will produce a gene model whether or not there is a gene there**, and nothing in the output distinguishes the two cases without a separate likelihood comparison against a null model (Module 03 §2).
- **And a predicted gene is a hypothesis**, which is **why annotation databases flag predicted versus experimentally supported entries — and why that distinction is routinely lost downstream**, when the identifier is cited without the flag.
- **Which is Module 03 §4's chain again: prediction → database annotation → cited fact, with the qualifier dropped at each step** — and this is the same mechanism as annotation transfer by similarity, with a different first step.

AND THE THING TO CARRY FORWARD

HMMs are the clearest example in this course of a principled probabilistic model — the score is a likelihood, which means it composes, can be compared across models, and has a meaning independent of the scale somebody chose, unlike an alignment score.

And the recurrence is Module 02's, which is why this module is short relative to its importance: **you already knew the algorithm** and only had to learn what the quantities mean.

Plus the general caution: a model that always returns an answer needs a separate test for 'there is nothing here' — which is the null model question (Module 03 §2) arriving in a third setting, and will arrive again in Module 10.

So: log space, labelled training wherever possible, and a null to compare against — three rules that cover most of what goes wrong with these models in practice.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Durbin et al., chapters 3 to 5 (library copy)	The whole module — and the profile HMM chapters are the definitive treatment.
Rabiner — A tutorial on hidden Markov models (free copies widely available)	§§1 and 2 — the three problems stated clearly, from speech recognition.
Eddy — What is a hidden Markov model? (free)	§1 in two pages — the best short explanation there is.
The HMMER documentation and Pfam (free)	§3 — profile HMMs as working software, and Pfam is the family database built from them.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Define an HMM** for a two-state toy problem.
- 2 **Run Viterbi by hand** on a five-symbol sequence.
- 3 **Implement Viterbi** in log space.
- 4 **Implement it without logs** and find the sequence length at which it underflows.
- 5 **Implement the forward algorithm** with log-sum-exp.

- 6 **Train with Baum-Welch** from three different initialisations.
- 7 **Build a profile HMM** from a small aligned family.
- 8 **Compare its hits** to BLAST's on the same database.
- 9 **Run an iterated search** and watch for drift.
- 10 **Run a gene finder on random sequence** and count the genes it predicts.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Define an HMM's components.
- 2 State the three standard problems.
- 3 Give the Viterbi recurrence and map it onto Needleman-Wunsch.
- 4 What is the relationship between Viterbi and forward?
- 5 State the two implementation rules and what each bug looks like.
- 6 Why does Baum-Welch need its initialisation reported?
- 7 What does a profile HMM capture that a matrix cannot?
- 8 Why do profiles find remote homologues, and how do they drift?
- 9 What does a gene finder always do, and why is that a problem?
- 10 State the three closing rules.